

Resume ATS Analysis - Model Architecture & Methodology

Model Evaluated: `src/model.py` (Sentence Transformers + FAISS) **Embedding Core:** `all-MiniLM-L6-v2`
Database Target: `data/final_cleaned.csv` & `data/job_embeddings.faiss`

1. Engine Architecture (Semantic Similarity)

Unlike rigid keyword-matching systems used in legacy Applicant Tracking Systems, this pipeline uses an advanced **Natural Language Processing (NLP) embedding matrix**. It evaluates the contextual "meaning" of a resume rather than just looking for exact word matches.

Text Extraction Pipeline

When a resume (PDF/DOCX) is uploaded via `app.py` :

1. Raw text is parsed utilizing `PyMuPDF` or `python-docx` .
 2. A generic parsing algorithm isolates the applicant's profile into three specific context windows:
 - `education_details`
 - `experience_details`
 - `skill`
 3. These windows are concatenated into a unified string representing the full candidate profile.
-

2. Core Model Vectors & FAISS Retrieval

The `all-MiniLM-L6-v2` transformer model maps the combined profile string into a **dense 384-dimensional mathematical vector**. Because evaluating exact 384-dimensional matches across thousands of jobs iterates slowly, the system utilizes **FAISS** (Facebook AI Similarity Search) to rapidly locate nearest neighbors.

1. **Vector Generation:** `model.encode(resume_text, convert_to_numpy=True)`
2. **L2 Normalization:** Vectors are squashed to a unit length `faiss.normalize_L2()` .
3. **IndexFlatIP Operation:** The FAISS index takes the Inner Product (IP) of L2 Normalized vectors, which mathematically equals perfectly scaled **Cosine Similarity**.

Mathematical Formulation

The Cosine Similarity value falls between `-1.0` and `1.0` . The system retrieves the Top 5 jobs that have the highest cosine similarity to the candidate's resume vector.

3. Algorithm Scoring & Metrics

The ATS system evaluates two distinct matching scenarios with different mathematical scoring baselines.

Scenario A: Job Board Matching (Top Jobs)

When matching the resume against the scraped job market database:

```
# System Formula
ATS_Score = (Cosine_Similarity * 100) + 10
```

(An offset of `+10` is given arbitrarily to normalize scores for display, pushing typical 50%-70% semantic similarities up into a friendlier 60%-80% "match" bracket).

Scenario B: Custom Job Description Matching

When an employer specifically provides a custom `jobDesc` text:

```
# System Formula
Custom_ATS_Score = (Cosine_Similarity * 100) + 30
```

(The `+30` scalar is applied heavily here. Because a custom Job Description is typically drastically shorter than an entire dataset aggregate, baseline semantic cosines sit lower. The `+30` boost bridges the metric up to human-readable ATS percentage scales).

4. Evaluation Limits & Optimization Options

- **Accuracy Metric:** Because this is an unsupervised semantic search algorithm, traditional "Accuracy %" (Precision/Recall) does not apply unless tested against a labeled ground-truth pairing dataset (e.g., historical hired/rejected candidate pairs).
- **Length Constraints:** The `all-MiniLM-L6-v2` model truncates inputs at 256 tokens. Extremely dense resumes might experience semantic "cutoff" at the bottom of the page (usually dropping older experiences).
- **Score Saturation:** With the manual `+30` scalar on custom matches, it is mathematically possible (though highly improbable) for a perfect semantic match to return a score greater than `100%`.